Pontificia Universidad Javeriana

Facultad de Ingeniería

Sistemas Operativos

Taller de Evaluación de Rendimiento

Martín Sanmiguel Delgado

Juan Manuel Baez Parra

Mateo Traslaviña Moreno

27 de abril. 2026

Bogotá - Colombia.

1. Introducción

El presente taller de evaluación de rendimiento tiene como objetivo comparar el comportamiento de distintos sistemas de cómputo, operando bajo distribuciones de Linux, al ejecutar diferentes algoritmos de multiplicación de matrices. El propósito central es contrastar el rendimiento de estas operaciones cuando se ejecutan de forma secuencial frente a implementaciones concurrentes (en paralelo), para posteriormente realizar un análisis a profundidad que correlacione el tiempo de ejecución con las especificaciones de hardware de cada máquina. Para garantizar la validez estadística de este análisis, se utilizará como métrica base el tiempo de ejecución promedio obtenido tras 30 iteraciones en cada escenario, a partir del cual se evaluarán indicadores clave como el *Speedup* (aceleración) y la eficiencia en la utilización de los recursos del sistema.

2. Descripción de los sistemas de computo

Para la realización del experimento se utilizaron 6 sistemas de cómputo Linux, con cuatro arquitecturas de hardware diferentes, ya que hay dos pares de máquinas que son iguales. A continuación, se muestran los componentes de hardware de cada uno de los equipos utilizados:

ID	CPU	Nucleos logicos	Frec. Máx. (GHz)	L1d/L1i (KiB)	L2 (MiB)	L3 (MiB)	RAM (GiB)	Tipo
PC-1	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	Portátil
PC-2	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	Portátil
Escritorio- 1	Intel Core i7-6700T	8	3.6	128/128	1.0	8	7.8	Desktop
Escritorio- 2	Intel Core i7-6700T	8	3.6	128/128	1.0	8	7.8	Desktop

PC-Martin	Intel Core i7-13650HX	20	4.9	544/704	11.5	24	14.8	Portátil
VM	Intel Xeon Gold 6240R (VMware)	4 (vCPU)	2.4	128/128	4.0	35.8	12	VM

Además, todas las maquinas que utilizamos tenían sistema operativo Linux Ubuntu

3. Diseño experimental

Para la realización del experimento se utilizaron dos algoritmos de multiplicacion de matrices los cuales fueron los siguientes:

- **Filas por columnas (FxC):** Es el método tradicional de multiplicar matrices. el cual consiste en multiplicar cada elemento de la fila de la primera matriz por cada elemento de la columna de la segunda matriz. Sin embargo, computacionalmente es ineficiente debido a la forma en que el lenguaje C guarda los datos en la memoria un principio conocido como Row-Major Order o almacenamiento por filas [1] [2]. Al recorrer la segunda matriz por columnas, el procesador se ve obligado a "saltar" constantemente entre diferentes direcciones de memoria [1], Esto se puede ver más fácilmente en la figura 1 que nos muestra cómo es que realmente los datos de los arreglos multidimensionales son almacenados. Esto hace que la memoria caché no se aproveche [3], ya que se viola el principio de localidad espacial, obligando al sistema a buscar los datos en la memoria principal, lo cual hace que el algoritmo sea mucho más lento, asi que podríamos esperar tiempos mucho mayores de este algoritmo.

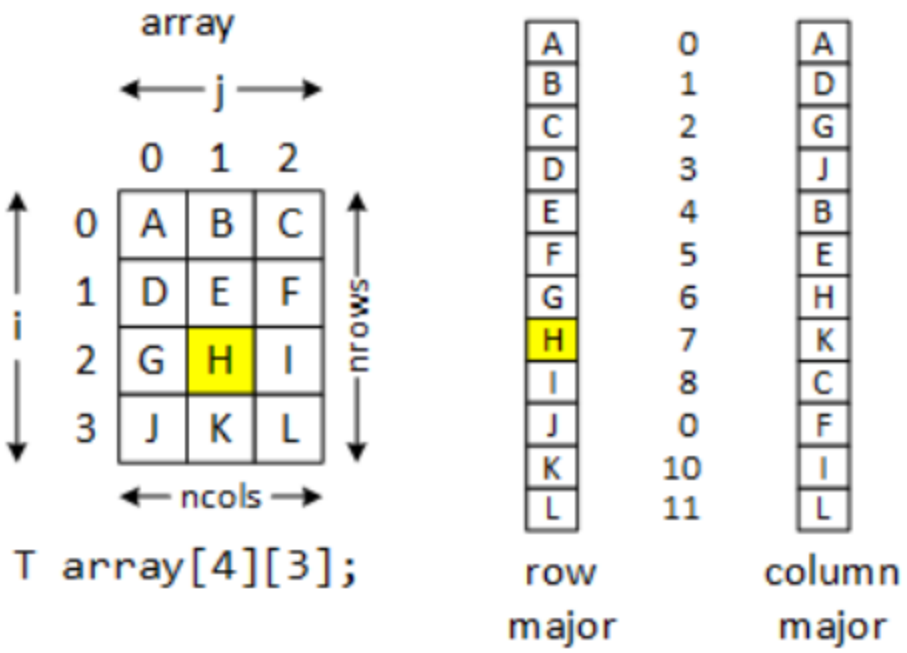

Figura 1: imagen tomada de [3] que representa Row-Major Order o almacenamiento por filas demostrando como el lenguaje C, almacena los datos de los arreglos multidimensionales

- **Filas por transpuestas (FxT):** Es una optimización pensada en cómo funciona la memoria del computador. Al transponer primero la segunda matriz, sus columnas se convierten en filas, haciendo que no se tenga que "saltar" constantemente entre direcciones de memoria aprovechando mucho mejor el principio de localidad espacial como se puede observar en la figura 1. De esta manera, cuando el procesador hace la multiplicación, lee ambas matrices de corrido, en línea recta. Todo esto genera que la memoria caché trabaja a su máxima velocidad, asi que podemos esperar un tiempo de ejecucion mucho menor comparandolo con el algoritmo Filas por columnas.

Antes de detallar las variables experimentales, es imperativo establecer la distinción a nivel de sistema operativo entre los dos mecanismos de concurrencia evaluados:

Hilo: Es una secuencia de instrucciones que el procesador ejecuta. Estos compranten memoria y necen de la necesidad de los programas de hacer varias cosas a la vez.

Proceso: Es un programa completo en ejecución. A diferencia de los hilos, los procesos son totalmente independientes y no comparten memoria.

Adicionalmente, para evaluar el comportamiento de estos algoritmos bajo distintas cargas de trabajo y niveles de concurrencia, el diseño del experimento contempla las siguientes variables:

- **Tamaño de las matrices:** Se realizarán pruebas con matrices cuadradas de dimensiones 512, 1024 y 2048.
- **Nivel de concurrencia:** Se evaluará una ejecución puramente secuencial 1 hilo y ejecuciones paralelas distribuyendo la carga de trabajo en 4, 8 y 16 hilos o procesos.
- **Mecanismos de paralelismo:** La concurrencia se abordará mediante dos implementaciones a nivel de sistema operativo: instanciación de **Hilos (POSIX)**, los cuales comparten el mismo espacio de direccionamiento de memoria, y creación de **Procesos (Fork)**, los cuales generan espacios de memoria independientes para cada bifurcación.

4. Recolección y Procesamiento de la Información

Dado que los sistemas operativos de propósito general ejecutan múltiples procesos en segundo plano que pueden alterar el rendimiento de la CPU en un momento dado, por lo tanto, se realizó el experimento varias veces para mitigar este tipo de variaciones:

Para garantizar la integridad y repetibilidad del experimento, la recolección se dividió en dos fases automatizadas:

Fase 1: Ejecución (Script Perl): Se utilizó el script lanzador.pl para automatizar la ejecución de toda la batería de pruebas en las cuatro máquinas. El script se encargó de compilar los binarios y ejecutar cada combinación exacta de variables (algoritmo, tamaño de matriz e hilos) 30 veces consecutivas. Los tiempos individuales de cada iteración se exportaron automáticamente a archivos de texto plano (.dat), generando un gran volumen de datos crudos.

Fase 2: Consolidación y visualización (Script Python): Para evitar errores de manipulación manual humana, se desarrolló un script de procesamiento de datos en Python utilizando las librerías Pandas, Seaborn y Matplotlib. Este código recorrió el árbol de directorios de las 4 máquinas, leyó cada archivo .dat y calculó el promedio aritmético exacto de las 30 iteraciones. Finalmente, el programa exportó los datos consolidados a archivos CSV y genero automáticamente las gráficas comparativas que sustentan el análisis posterior.

5. Análisis de Resultados

En esta sección se evalúa el impacto de las distintas variables de hardware y de software sobre los tiempos de ejecución. El análisis se fundamenta en el contenido obtenido por los Scripts descritos en la sección anterior

5.1. Impacto de la carga de trabajo y escalabilidad

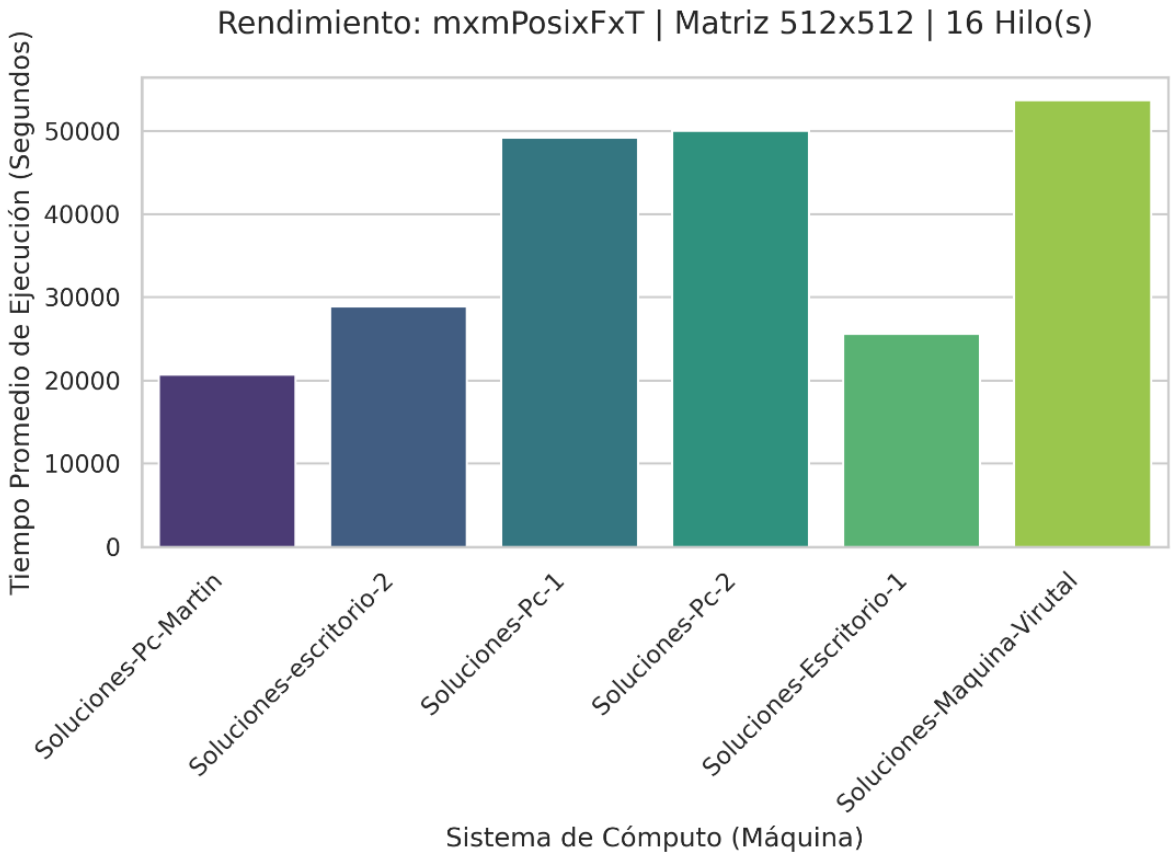

Figura 2: Grafica de barras que muestra el tiempo de ejecución de cada una de las maquinas al realizar una multiplicacion de matrices de tamaño 512x512 utilizando POSIX y 16 hilos

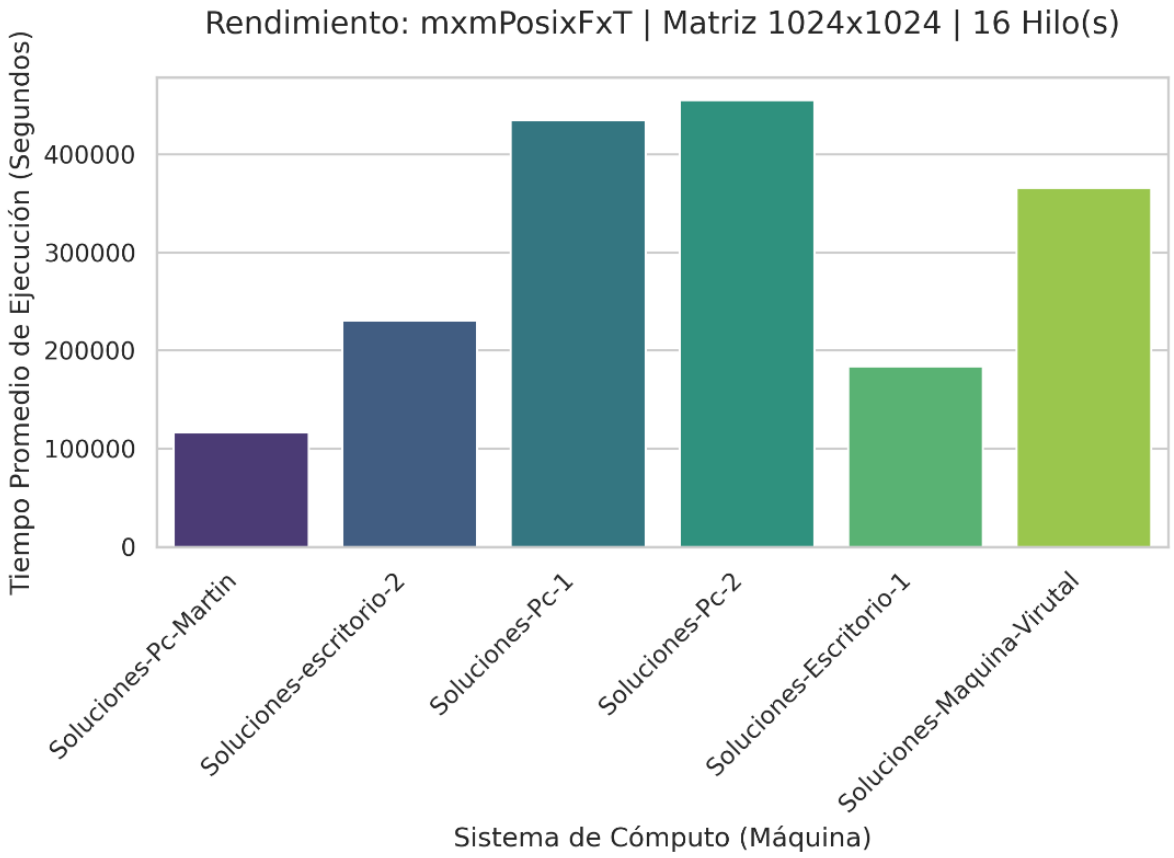

Figura 3: Grafica de barras que muestra el tiempo de ejecución de cada una de las maquinas al realizar una multiplicacion de matrices de tamaño 1024x1024 utilizando POSIX y 16 hilos

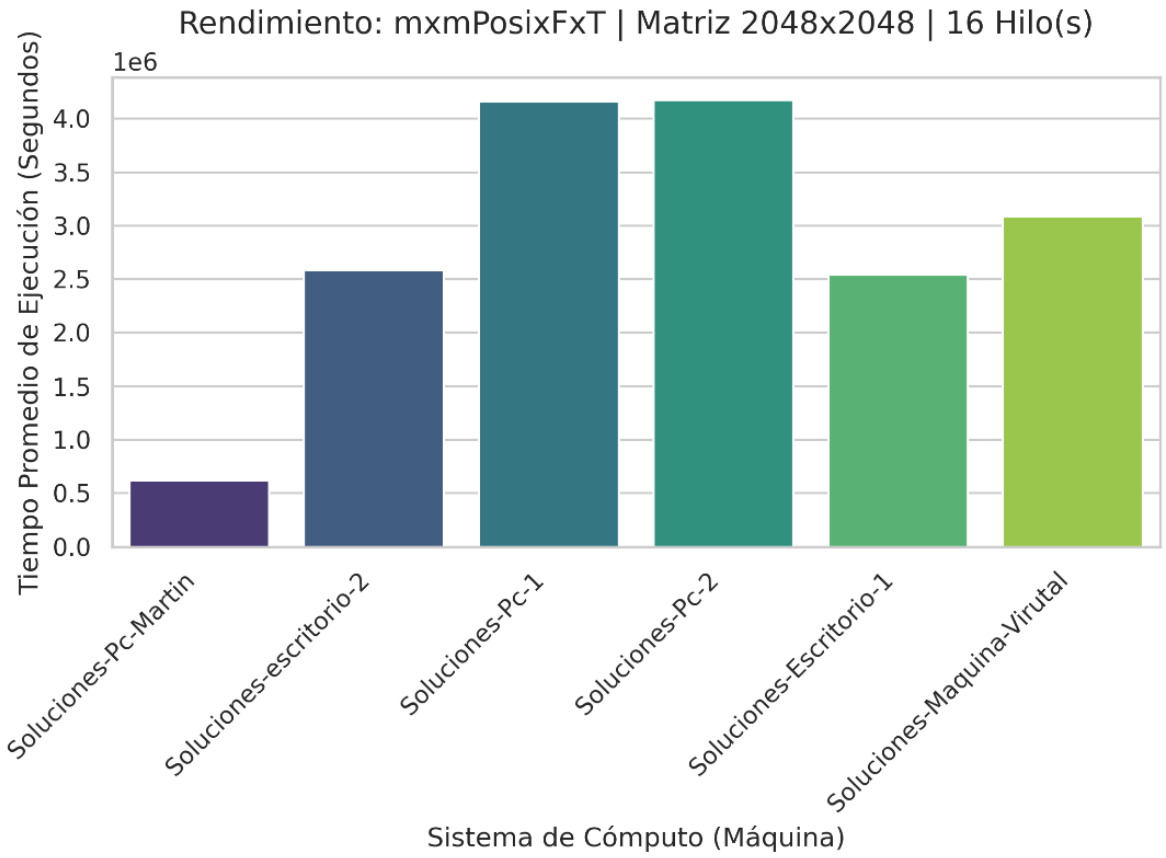

Figura 4: Grafica de barras que muestra el tiempo de ejecución de cada una de las maquinas al realizar una multiplicación de matrices de tamaño 2048x2048 utilizando POSIX y 16 hilos

Cuando se analizó por primera vez estas graficas se creyo que se habia realizado de forma errónea el experimento ya que a primera vista “no hace sentido”, ya que por sentido común es normal pensar que a mayor cantidad de datos mayor tiempo de ejecución, pero si prestamos atencion podemos ver qué es exactamente lo opuesto y esto se debe a un concepto llamado Overhead,

REFERENCIAS

[1] Weber State University, "Row-Major Order," *CS 1410 Textbook*. [En línea]. Disponible en: https://icarus.cs.weber.edu/~dab/cs1410/textbook/7.Arrays/row_major.html. [Accedido: 27-abr-2026].

[2] Wikipedia, "Row- and column-major order," *Wikipedia, The Free Encyclopedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Row-_and_column-major_order. [Accedido: 27-abr-2026].

[3] W. Stallings, *Organización y arquitectura de computadores*, 8va ed. Pearson Educación, 2012, cap. 4. [Accedido: 27-abr-2026].